Brief Announcement: Content-Oblivious Leader Election on Rings

Fabian Frei
CISPA Helmholtz Center for Information Security
Saarbrücken, Germany
fabian.frei@cispa.de

Ran Gelles
Bar-Ilan University
Ramat Gan, Israel
ran.gelles@biu.ac.il

Ahmed Ghazy
CISPA Helmholtz Center for Information Security
Saarbrücken, Germany
ahmed.ghazy@cispa.de

Alexandre Nolin
CISPA Helmholtz Center for Information Security
Saarbrücken, Germany
alexandre.nolin@cispa.de

ABSTRACT

In *content-oblivious computation*, n nodes wish to compute a given task over an asynchronous network that suffers from an extremely harsh type of noise, which corrupts the content of all messages across all channels. In a recent work, Censor-Hillel, Cohen, Gelles, and Sela (Distributed Computing, 2023) showed how to perform arbitrary computations in a content-oblivious way in 2-edge connected networks but only if the network has a distinguished node (called *root*) to initiate the computation.

Our goal is to remove this assumption, which was conjectured to be necessary. Achieving this goal essentially reduces to performing a content-oblivious leader election since an elected leader can then serve as the root required to perform arbitrary content-oblivious computations. We focus on ring networks, which are the simplest 2-edge connected graphs. On *oriented* rings, we obtain a leader election algorithm with message complexity $O(n \cdot \mathsf{ID}_{\max})$, where $\mathsf{ID}_{\max}$ is the maximal assigned ID. As it turns out, this dependency on $\mathsf{ID}_{\max}$ is inherent: we show a lower bound of $\Omega(n \log(\mathsf{ID}_{\max}/n))$ messages for content-oblivious leader election algorithms. We also extend our results to *non-oriented* rings, where nodes cannot tell which channel leads to which neighbor. In this case, however, the algorithm does not terminate but only reaches quiescence.

CCS CONCEPTS

• **Theory of computation** → Distributed algorithms; Distributed computing models; • **Networks** → Network reliability; Ring networks.

KEYWORDS

Leader Election, Ring Networks, Content-Oblivious Computation, Fully Defective Networks

ACM Reference Format:
Fabian Frei, Ran Gelles, Ahmed Ghazy, and Alexandre Nolin. 2024. Brief Announcement: Content-Oblivious Leader Election on Rings. In *ACM Symposium on Principles of Distributed Computing (PODC '24), June 17–21, 2024, Nantes, France.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3662158.3662785

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
PODC '24, June 17–21, 2024, Nantes, France
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0668-4/24/06
https://doi.org/10.1145/3662158.3662785

1 INTRODUCTION

The field of distributed computing is rich with models helping us understand different types of architectures and computational hardness by making different kinds of assumptions about how computations are carried out. A recent work by Censor-Hillel, Cohen, Gelles, and Sela [6] introduced a particularly weak computational model coined *fully defective networks*. This model considers an asynchronous network in which messages may be fully corrupted and thus not carry any information beyond their sheer existence. Algorithms designed for this model cannot rely in any way on possible contents of messages, and thus are named *content-oblivious*. Instead of relying on the content of messages or on their time of arrival (since arbitrary delays may occur in asynchronous networks), such algorithms depend solely on the *order* in which messages arrive from different neighbors.

The aforementioned work by Censor-Hillel et al. [6] showed that any computation possible in the asynchronous setting with reliable content-carrying messages can be simulated in the fully defective setting under two assumptions: that the network is 2-edge connected and that there is a distinguished leader (which they call the *root node*) in the network. While the same paper showed 2-edge connectivity to be essential for any kind of nontrivial computation in fully defective networks, the question of the necessity of a pre-existing leader was not settled. Censor-Hillel et al. conjectured that general computations in fully defective networks do indeed require the existence of such a pre-elected leader.

In this paper, we *disprove* this conjecture, at least for the most fundamental type of 2-edge connected topologies, namely, rings. We design a content-oblivious algorithm that successfully performs a leader election in oriented rings.

Theorem 1. *There is a* quiescently terminating *content-oblivious algorithm of message complexity* $n(2 \cdot \mathsf{ID}_{\max} + 1)$ *that elects a leader in oriented rings of* n *nodes with unique IDs.*

Here, $\mathsf{ID}_{\max}$ denotes the maximal ID assigned to a node in the network, and *quiescent termination* refers to the valuable property of our algorithm that nodes do not receive messages after termination.

Quiescent termination enables the combination of our content-oblivious leader election algorithm with the root-dependent universal content-oblivious algorithm [6, Thm. 1]. This yields the following powerful corollary.

Corollary 2. *Assuming unique IDs, any asynchronous algorithm on rings can be simulated in a fully defective oriented ring.*

The term $\mathsf{ID}_{\max}$ appearing in Theorem 1 is unusual for the message complexity of an algorithm.[1] Somewhat surprisingly, we can show that this term is inherent to content-oblivious algorithms.

Theorem 3. *Any deterministic terminating content-oblivious algorithm for leader election in rings with unique IDs sends at least* $n\lfloor \log_2(\mathsf{ID}_{\max}/n) \rfloor$ *messages.*

Note that this $\Omega(n \log(\mathsf{ID}_{\max}/n))$ lower bound implies that the number of messages in a ring of size n is unbounded—we can always increase the message complexity by assigning larger IDs, even when n is a small constant.

In this brief announcement, we present the leader election algorithm behind Theorem 1 in Section 4 and provide some intuition for its correctness. The complete proof of correctness, the entire discussion of our lower bound result, and additional results for non-oriented rings are found in the full version of this paper [19].

2 RELATED WORK

Leader election is a fundamental task that has been studied by the distributed computing community since the 1970s. Simple leader election algorithms for asynchronous rings were proposed by Le Lann [24] and by Chang and Roberts [8]. These algorithms employ $O(n^2)$ messages to ensure that all nodes yield to the node with the maximal ID, which becomes the leader. Later work [12, 22, 25] improved this down to $O(n \log n)$ messages. This complexity is tight in asynchronous rings [5, 18]. In *synchronous* rings, leader election can be performed by communicating only $O(n)$ messages [15, 18].

For the case of anonymous rings, i.e., with identical nodes without IDs, Angluin [2] proved that symmetry cannot be broken, and thus no leader election algorithm exists. Attiya, Snir, and Warmuth [4] examined anonymous asynchronous rings further and characterized computable tasks. In particular, they showed that many tasks, including ring orientation, require communicating $\Omega(n^2)$ messages. Flocchini et al. [16] presented algorithms for orienting a ring and leader election when n is known to the nodes.

The question of termination in anonymous rings was explored by Itai and Rodeh [23], who discovered that a network cannot compute its size n by a terminating algorithm. As a consequence, a leader election algorithm that terminates is impossible, too, since it is trivial to learn n once a leader is given. On the other hand, if the nodes know n or even an upper bound on n, a randomized leader election algorithm that terminates exists [23]. Afek and Matias [1] give various leader election algorithms that trade off knowledge, termination, and number of sent messages.

A content-oblivious BFS algorithm by Censor-Hillel, Gelles, and Haeupler [7] is a pre-processing step for their distributed interactive coding scheme [20]. Building on that idea, Censor-Hillel et al. [6] introduced the concept of content-oblivious computation and proved that general computations are only possible over 2-edge connected networks. They designed a compiler that converts any asynchronous algorithm into a content-oblivious one, assuming a pre-existing leader.

Computation with fully corrupted messages has been studied mainly in the synchronous setting, where the presence or absence of a message in a given round can still be used to send one bit of information [26]. A notable example is the *Beeping* model, introduced by Cornejo and Kuhn [9], in which each message sent by a node is received by all its neighbors, and nodes can only distinguish between receiving no message and at least one message. Leader election has been studied in this setting in a sequence of works [10, 13, 17, 21]. Similar to the fully defective setting, compilers were devised for transforming algorithms with stronger communication primitives into algorithms for the Beeping model [3, 11, 14].

[1] Some algorithms depend on the IDs assigned to nodes. However, it is common for the complexity analysis to assume that all IDs are of length $O(\log n)$ bits, making the dependence on the IDs implicit.

3 PRELIMINARIES

The content-oblivious computation model. Consider a distributed network $G = (V, E)$ with $n = |V|$ nodes. We usually assume that each node $v \in V$ is assigned a unique ID, usually denoted by $\mathsf{ID}_v \in \mathbb{N}$. We denote the largest ID assigned to a node in the network by $\mathsf{ID}_{\max}$, that is, $\mathsf{ID}_{\max} := \max_{v \in V} \mathsf{ID}_v$, and by ℓ the node possessing this ID. Apart from the IDs, the nodes are identical. Our algorithms are *uniform*, i.e., the nodes neither know the size of the network n nor any bound on it.

In this paper, we consider rings, i.e., connected graphs where all nodes have degree 2. Neighboring nodes communicate by sending messages to each other. We assume that all messages are subject to corruption: the content of any message is completely erased by noise, resulting in an empty message of length 0, which we call a *pulse*. Computations that ignore any potential content of a message and thus work only with pulses are called *content-oblivious*.

Further, the network is *asynchronous*: the time it takes a pulse to travel through a channel and arrive at its end is unpredictable; the delays are unbounded but always finite. Pulses cannot be dropped or injected by the channel. In such an asynchronous network, the nodes possess neither a common clock nor any notion of time, and they are assumed to be *event-driven*. This means that a node may act once right in the beginning of the computation and from then on only upon receiving a pulse. As a function of its own ID, the previously received pulses, and possibly its own source of randomness, it can then change its state and, for each connected channel, send any number of pulses. The algorithm presented in this brief announcement is deterministic, but we also make use of randomness for some algorithms in the full version of this paper [19]. We say that an algorithm *terminates* (sometimes also referred to as process termination, explicit termination, or termination detection in the literature) if for each node there is a time at which it has decided on an output and entered a terminating state. Once in a terminating state, a node ignores all incoming pulses and does not send any new ones. We say that an algorithm has *quiescent termination* if at the time the last node terminates it is guaranteed that no pulses are still in transit. An algorithm's *message complexity* is the total number of messages (pulses) it sends during a worst-case computation until all nodes have terminated or until the network has reached quiescence.

Ring's orientation. In a ring, each node communicates with its two neighbors via Port_0 and Port_1. Consider a pulse re-sent from Port_1 by every node receiving it. If such a pulse passes through all edges (i.e., is never reflected by a node with misaligned ports), we call the ring *oriented* and the pulse *clockwise* (CW). Port_1 of each node is then called its CW port, leading to its CW neighbor. Counterclockwise (CCW) is defined analogously via Port_0.

4 LEADER ELECTION IN AN ORIENTED RING

Recall that in the *leader election task*, nodes are required to terminate with an output: Leader or Non-Leader, where a single node $\ell \in V$ outputs Leader, while all other nodes must output Non-Leader. We design an algorithm to elect a leader in an oriented ring of unknown size that features quiescent termination. We will use the following methods for sending and receiving pulses. The method sendCW() sends one pulse over the CW channel. The method recvCW() checks whether pulses are waiting in the CW incoming queue. If no pulse is in the queue, the method returns 0. Otherwise, it consumes a single pulse from the queue and returns 1. The methods sendCCW() and recvCCW() are the analogous CCW versions. Moreover, for each node v, we introduce counters ρ_{cw} and σ_{cw} for the total number of received and sent CW pulses, respectively. Likewise, we introduce ρ_{ccw} and σ_{ccw} for CCW pulses. The methods update the counters with every received or sent pulse.

Algorithm 1 Quiescently Stabilizing Leader Election for Node v

```
1: sendCW()
2: while true do
3:     if recvCW() returns 1 then
4:         if ρ_cw = ID_v then
5:             state ← Leader
6:         else                          ▷ v acts as a relay unless ρ_cw = ID_v
7:             state ← Non-Leader
8:             sendCW()
```

Quiescent Stabilization. Algorithm 1 uses only clockwise pulses. Each node v sends one initial pulse, and then relays all received pulses, except the one that increases its counter of received pulses to its ID. When this event $\rho_{\text{cw}} = \text{ID}$ occurs, the node declares itself a leader. Its goes back to being a non-leader and relaying pulses if another pulse is received. We can prove that Algorithm 1 is a quiescently stabilizing algorithm for leader election. The main intuition is that each node adds a pulse to the clockwise circulation and removes one only as it declares itself a leader. Thus pulses keep circulating until eventually every node has declared itself leader at some point. Using suitable invariants, we can show that this happens last for the node of maximal ID and that all other nodes revert to being non-leaders before this occurs. The nodes can never be sure, however, whether the final leader is already elected and all pulses have been removed from the network. This is different for Algorithm 2, which achieves such a quiescent termination.

Quiescent Termination. Algorithm 2 interweaves two instances of Algorithm 1: one over the CW channel (Lines 3–8) and one over the CCW channel (Lines 9–13). The CW instance starts, as before, upon initialization, whereas the CCW instance starts for a node only once it tentatively declares itself the leader due to the CW instance. This guarantees that the CCW instance lags behind the CW one. Finally, Lines 14–17 identify an event unique to the node of maximal ID, namely as many received clockwise pulses and as many received counterclockwise pulses as the node's ID, and initiate a termination process. We can show that when the mentioned event occurs all nodes have received exactly $\text{ID}_{\max}$ clockwise pulses and exactly $\text{ID}_{\max}$ counterclockwise pulses. This happened last for the node of maximal ID. The unique event lets this node know that it is the final leader. It then sends one more CCW *termination pulse*. This pulse causes, for any node receiving it, for the first time the counter of received counterclockwise pulses to surpass the counter of received clockwise pulses. Any node that was a leader before but is no longer forwards the termination pulse and then terminates (Line 18). The termination pulse is thus forwarded until it returns to the leader, causing it to terminate without forwarding the pulse.

Algorithm 2 Quiescently Terminating Leader Election for Node v

```
1: sendCW()
2: repeat
3:     if recvCW() returns 1 then        ▷ Alg. 1 over CW channel
4:         if ρ_cw = ID_v then
5:             state ← Leader
6:         else
7:             state ← Non-Leader
8:             sendCW()
9:     if ρ_cw ≥ ID_v then    ▷ Lagging Alg. 1 over CCW channel
10:        if σ_ccw = 0 then sendCCW() end if
11:        if recvCCW() returns 1 then
12:            if ρ_ccw ≠ ID_v then
13:                sendCCW()
14:    if ρ_cw = ID_v = ρ_ccw then       ▷ Send termination pulse
15:        sendCCW()
16:        while recvCCW() returns 0 do
17:            pass          ▷ Wait for return of termination pulse
18: until ρ_ccw > ρ_cw
19: output state
```

Algorithm 2 has the message complexity of $n(2 \cdot \text{ID}_{\max}+1)$ stated in Theorem 1. In particular, the number of pulses by the algorithms increases indefinitely if we assign higher and higher IDs to the nodes. Our lower bound proves this behavior to be inevitable.

Lower Bound on Message Complexity. We provide here only a rough sketch of the proof of Theorem 3. We define for any given algorithm an ID's *solitude pattern*, which is a binary string describing the pattern of pulses sent out by a node of this ID in a ring without other nodes. A symmetry argument for rings with exactly two nodes shows that each ID's solitude pattern must be unique. The pigeon-hole principle is then applied to show that we can assign to the nodes of any given ring specific IDs with an arbitrarily long common prefix when choosing from a sufficiently large pool. With a scheduler delivering pulses in synchrony, the nodes are initially indistinguishable as they send pulses according to the common prefix, yielding Theorem 3.

Full Paper. The complete algorithm analysis, all proof details for the lower bound, and further results are found in the full paper [19].

ACKNOWLEDGMENTS

R. Gelles is partially supported by the United States-Israel Binational Science Foundation (BSF), Jerusalem, Israel, Grant No. 2020277. He would like to thank the CISPA Helmholtz Center for Information Security for hosting him while part of this research was done.

REFERENCES

[1] Y. Afek and Y. Matias. 1994. Elections in Anonymous Networks. *Information and Computation* 113, 2 (1994), 312–330. https://doi.org/10.1006/inco.1994.1075

[2] Dana Angluin. 1980. Local and Global Properties in Networks of Processors (Extended Abstract). In *Proceedings of the Twelfth Annual ACM Symposium on Theory of Computing* (Los Angeles, California, USA) *(STOC '80)*. ACM, New York, NY, USA, 82–93. https://doi.org/10.1145/800141.804655

[3] Yagel Ashkenazi, Ran Gelles, and Amir Leshem. 2022. Noisy beeping networks. *Information and Computation* 289, A (2022), 104925. https://doi.org/10.1016/J.IC.2022.104925

[4] Hagit Attiya, Marc Snir, and Manfred K. Warmuth. 1988. Computing on an Anonymous Ring. *J. ACM* 35, 4 (oct 1988), 845–875. https://doi.org/10.1145/48014.48247

[5] James E. Burns. 1980. *A formal model for message passing systems.* Technical Report. Computer Science Dept., Indiana Univ., Bloomington, Ind. TR91.

[6] Keren Censor-Hillel, Shir Cohen, Ran Gelles, and Gal Sela. 2023. Distributed computations in fully-defective networks. *Distributed Computing* 36, 4 (2023), 501–528. https://doi.org/10.1007/s00446-023-00452-2

[7] Keren Censor-Hillel, Ran Gelles, and Bernhard Haeupler. 2019. Making asynchronous distributed computations robust to noise. *Distributed Computing* 32, 5 (01 Oct 2019), 405–421. https://doi.org/10.1007/s00446-018-0343-5

[8] Ernest Chang and Rosemary Roberts. 1979. An Improved Algorithm for Decentralized Extrema-Finding in Circular Configurations of Processes. *Commun. ACM* 22, 5 (may 1979), 281–283. https://doi.org/10.1145/359104.359108

[9] Alejandro Cornejo and Fabian Kuhn. 2010. Deploying Wireless Networks with Beeps. In *Distributed Computing, 24th International Symposium, DISC 2010 (LNCS, Vol. 6343)*. Springer, 2010, Cambridge, MA, USA, September 13–15, 2010. Proceedings, 148–162. https://doi.org/10.1007/978-3-642-15763-9_15

[10] Artur Czumaj and Peter Davies. 2019. Leader election in multi-hop radio networks. *Theoretical Computer Science* 792 (2019), 2–11. https://doi.org/10.1016/J.TCS.2019.02.027

[11] Peter Davies. 2023. Optimal Message-Passing with Noisy Beeps. In *Proceedings of the 2023 ACM Symposium on Principles of Distributed Computing, PODC*. ACM, 2023, Orlando, FL, USA, June 19–23, 2023, 300–309. https://doi.org/10.1145/3583668.3594594

[12] Danny Dolev, Maria Klawe, and Michael Rodeh. 1982. An $O(n \log n)$ unidirectional distributed algorithm for extrema finding in a circle. *Journal of Algorithms* 3, 3 (1982), 245–260. https://doi.org/10.1016/0196-6774(82)90023-2

[13] Fabien Dufoulon, Janna Burman, and Joffroy Beauquier. 2018. Beeping a Deterministic Time-Optimal Leader Election. In *32nd International Symposium on Distributed Computing, DISC 2018 (LIPIcs, Vol. 121)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, New Orleans, LA, USA, October 15–19, 2018, 20:1–20:17. https://doi.org/10.4230/LIPICS.DISC.2018.20

[14] Fabien Dufoulon, Janna Burman, and Joffroy Beauquier. 2020. Can Uncoordinated Beeps tell Stories?. In *PODC '20: ACM Symposium on Principles of Distributed Computing*. ACM, Virtual Event, Italy, August 3–7, 2020, 408–417. https://doi.org/10.1145/3382734.3405699

[15] M. El-Ruby, J. Kenevan, R. Carlson, and K. Khalil. 1991. A linear algorithm for election in ring configuration networks. In *Proceedings of the Twenty-Fourth Annual Hawaii International Conference on System Sciences*, Vol. i. IEEE Computer Society, Los Alamitos, CA, USA, 117–123 vol.1. https://doi.org/10.1109/HICSS.1991.183877

[16] Paola Flocchini, Evangelos Kranakis, Danny Krizanc, Flaminia L. Luccio, and Nicola Santoro. 2004. Sorting and election in anonymous asynchronous rings. *J. Parallel and Distrib. Comput.* 64, 2 (2004), 254–265. https://doi.org/10.1016/j.jpdc.2003.11.007

[17] Klaus-Tycho Förster, Jochen Seidel, and Roger Wattenhofer. 2014. Deterministic Leader Election in Multi-hop Beeping Networks - (Extended Abstract). In *Distributed Computing – 28th International Symposium, DISC'14 (LNCS, Vol. 8784)*. Springer, 2014, Austin, TX, USA, October 12–15, 2014. Proceedings, 212–226. https://doi.org/10.1007/978-3-662-45174-8_15

[18] Greg N. Frederickson and Nancy A. Lynch. 1987. Electing a Leader in a Synchronous Ring. *J. ACM* 34, 1 (jan 1987), 98–115. https://doi.org/10.1145/7531.7919

[19] Fabian Frei, Ran Gelles, Ahmed Ghazy, and Alexandre Nolin. 2024. Content-Oblivious Leader Election on Rings. *CoRR* abs/2405.03646 (2024). https://arxiv.org/abs/2405.03646

[20] Ran Gelles. 2017. Coding for Interactive Communication: A Survey. *Foundations and Trends® in Theoretical Computer Science* 13, 1–2 (2017), 1–157. https://doi.org/10.1561/0400000079

[21] Mohsen Ghaffari and Bernhard Haeupler. 2013. Near Optimal Leader Election in Multi-Hop Radio Networks. In *SODA'13*. SIAM, New Orleans, Louisiana, USA, January 6–8, 2013, 748–766. https://doi.org/10.1137/1.9781611973105.54

[22] D. S. Hirschberg and J. B. Sinclair. 1980. Decentralized Extrema-Finding in Circular Configurations of Processors. *Commun. ACM* 23, 11 (nov 1980), 627–628. https://doi.org/10.1145/359024.359029

[23] Alon Itai and Michael Rodeh. 1990. Symmetry breaking in distributed networks. *Information and Computation* 88, 1 (1990), 60–87. https://doi.org/10.1016/0890-5401(90)90004-2

[24] Gérard Le Lann. 1977. Distributed Systems - Towards a Formal Approach. In *Information Processing, Proceedings of the 7th IFIP Congress 1977*, Bruce Gilchrist (Ed.). North-Holland, Toronto, Canada, August 8–12, 1977, 155–160.

[25] Gary L. Peterson. 1982. An $O(n \log n)$ Unidirectional Algorithm for the Circular Extrema Problem. *ACM Trans. Program. Lang. Syst.* 4, 4 (oct 1982), 758–762. https://doi.org/10.1145/69622.357194

[26] Nicola Santoro and Peter Widmayer. 1990. Distributed Function Evaluation in the Presence of Transmission Faults. In *Algorithms, International Symposium SIGAL '90 (Lecture Notes in Computer Science, Vol. 450)*, Tetsuo Asano, Toshihide Ibaraki, Hiroshi Imai, and Takao Nishizeki (Eds.). Springer, Tokyo, Japan, August 16–18, 1990, 358–367. https://doi.org/10.1007/3-540-52921-7_85